

Multimodal AI for Oncology

Pipeline Documentation

Image Feature Extraction · Clinical Text Embeddings · Late Fusion

Bahar Sevgin

Queen Mary University of London — Maiques Lab

Overview	
Dataset	177 melanocytic skin lesion cases — paired dermoscopy images + pathology reports
Classes	Dysplastic Nevus (DN) · Melanoma Stage IA (MIA) · Melanoma In Situ (Minsitu)
Image model	PanDerm Large (ViT-L/16) — fine-tuned, 1024-dim CLS features
Text model	BioClinicalBERT — mean-pooled, 768-dim embeddings
Fusion	Concatenation (1792-dim) → Logistic Regression, 5-fold CV
Best result	Balanced Accuracy 0.603 ± 0.060 · Macro AUC 0.778 ± 0.057

1. Pipeline Overview

The pipeline processes two parallel data streams — dermoscopy images and clinical pathology reports — through separate deep learning encoders, then combines the resulting feature vectors via late fusion for 3-class classification.

Image branch stages:

PREPARE	SEGMENT	MAKE CSV	FINE-TUNE	EXTRACT	EVALUATE
Manifest + k-fold splits	LAB + Otsu lesion crop	PanDerm fold CSVs	PanDerm ViT-L/16	1024-dim CLS features	LogReg + plots

Text branch stages:

LOAD REPORTS	PREPROCESS	BIOCLINICALBERT	SAVE	VALIDATE
Excel / CSV input	Concatenate fields	Mean-pool tokens	clinical_embeddings.npy	UMAP + kNN LOO

Late fusion: both branches feed into a single logistic regression.

Script Responsibilities

Script	Stage	GPU?	Output
prepare_data.py	1 — Manifest	No	dataset_manifest.csv
segment_lesions.py	2 — Segmentation	No	segmented_cache/
make_panderm_csv.py	3 — Fold CSVs	No	cross-fold-csv/
panderm_finetuning.py	4 — Fine-tuning	Yes	results_fold{0-4}/
extract_features.py	5 — Feature extraction	Yes	features/*.npy
evaluate.py	6 — Evaluation + plots	No	results/*.png, *.csv
clinical_pipeline.py	7 — Text embeddings	No	clinical_embeddings.npy
late_fusion.py	8 — Late fusion	No	fusion_results/

All scripts share a single **config.py** at the repo root. Editing the four path variables there is the only setup step required on a new machine.

2. Data Preparation

2.1 Dataset Manifest — prepare_data.py

The script scans the dermoscopy directory (one sub-folder per class, one sub-sub-folder per patient containing JPEG/PNG images). It builds a flat DataFrame with columns: *image_path*, *patient_id*, *diagnosis*, *label*, *fold*.

Patient-level stratified k-fold splitting:

StratifiedGroupKFold (scikit-learn) is used with *groups=patient_id* and *y=diagnosis*. This guarantees two properties simultaneously:

- No patient appears in both train and test of the same fold (prevents data leakage from multi-image patients).
- Class proportions are approximately balanced across folds.

Fold	DN patients	MIA patients	Minsitu patients	Total images
0	10	12	10	109
1	9	13	11	117
2	11	12	11	119
3	15	13	7	121
4	11	9	12	96

3 patients appear under two diagnoses (e.g. DN and MIA), giving 166 patient groups from 163 unique patients. These multi-class patients are handled by aggregating features per (patient_id, diagnosis) group.

2.2 Lesion Segmentation — segment_lesions.py

Raw dermoscopy images contain black corner artefacts and skin surrounding the lesion. Segmentation crops to the lesion region so PanDerm focuses on the clinically relevant area.

Algorithm (colour-based, no deep learning required):

1. Convert image to LAB colour space.
2. Apply Otsu thresholding on the L (luminance) channel.
3. Morphological closing (kernel 15×15) to fill gaps.
4. Extract the largest connected component as the lesion mask.
5. Crop to the bounding box of the mask with a 10% margin.
6. Fallback: if mask area < 1% of image, keep original image.

Outputs are cached to segmented_cache/. A QC montage image (segmentation_qc_montage.png) is generated for visual inspection.

3. Image Feature Extraction

PanDerm is a Vision Transformer foundation model pre-trained on 2 million dermatology images across 11 datasets. We use the Large variant (ViT-L/16) which processes 224×224 images as 14×14 = 196 non-overlapping 16×16 patches plus one special [CLS] token.

3.1 Model Architecture

Component	Detail
Architecture	ViT-L/16 (Vision Transformer Large, patch size 16)
Input	224 × 224 × 3 RGB image
Tokens	196 patch tokens + 1 [CLS] token = 197 total
Transformer depth	24 blocks
Attention heads	16 per block
Feature dimension	1024-dim per token
Output used	[CLS] token — global image representation
Normalisation	mean=[0.485,0.456,0.406], std=[0.228,0.224,0.225]
Pre-training	emilyalsentzer/Bio_ClinicalBERT weights on 2M derm images

3.2 Fine-Tuning Strategy

Although PanDerm is pre-trained on dermatology images, it has not seen our specific 3-class task (DN / MIA / Minsitu). Fine-tuning adapts the model to distinguish these subtle morphological differences.

Key training choices:

- Layer-wise learning rate decay (0.65)** — lower layers learn slowly (they already encode general visual features), upper layers learn faster (they need to specialise for our task).
- Mixup (0.8) + CutMix (1.0)** — data augmentation by interpolating or patching pairs of training images, regularising the model against overfitting on our small dataset (562 images).
- 5-fold cross-validation** — each fold produces its own checkpoint. Test images are always from a fold the model has never seen during training.

Hyperparameter	Value
Epochs	50 (warmup 5)
Batch size	32
Optimiser	AdamW
Layer decay	0.65
Drop path	0.2

Weight decay	0.05
Mixup alpha	0.8
CutMix alpha	1.0
Output classes	3 (DN, MIA, Minsitu)

3.3 Feature Extraction — `extract_features.py`

After fine-tuning, each fold's checkpoint is loaded and run in inference mode. The [CLS] token output of the final transformer block is used as the image representation — it aggregates global context from all 196 patches via self-attention.

Extraction pipeline per fold:

1. Load fine-tuned checkpoint with `weights_only=False` (PyTorch 2.x).
2. Call `model.forward_features(img, is_train=False)` → shape (1, 197, 1024).
3. Take index 0 → the [CLS] token → 1024-dim vector per image.
4. Save per-image features: `fold{i}_image_features.npy` — shape (562, 1024).
5. **Patient aggregation:** mean-pool features across all images of each (patient_id, diagnosis) group → (166, 1024).

Mean pooling at patient level is important because most patients have 3–5 images of the same lesion. Averaging reduces noise from imaging variation and gives one stable representation per patient per diagnosis.

4. Clinical Text Embeddings

Pathology reports contain structured clinical knowledge — diagnosis, macroscopic description, excision margins, ulceration, mitosis — that is complementary to the visual features captured by PanDerm. We encode reports with BioClinicalBERT, a BERT model trained on MIMIC-III clinical notes.

4.1 Model

Property	Detail
Model	emilyalsentzer/Bio_ClinicalBERT
Pre-training	BERT-base architecture trained on MIMIC-III clinical notes
Vocabulary	Clinical terminology, drug names, medical abbreviations
Input	Up to 512 sub-word tokens
Output	768-dim embedding per token (last hidden state)
Pooling	Mean of all token embeddings (excluding [CLS] and [SEP])
Final dim	768 per patient

4.2 Report Construction

Each patient's report is constructed by concatenating two structured fields:

- **diagnostic_summary_english** — final pathology diagnosis.
- **macroscopic_description_english** — gross description of the lesion sample.

Reports were originally in English, Spanish, and Catalan. All were translated to English before embedding to ensure a consistent token distribution. Long reports are truncated to 512 tokens.

4.3 Structured Feature Flags

In addition to dense embeddings, six binary clinical flags are extracted via regular expressions as an interpretable complement to the embeddings:

Flag	Trigger keywords
flag_complete_excision	complete excision, free margins, free of lesion
flag_ulceration	ulceration, ulcerated
flag_mitosis	mitosis, mitotic
flag_vascular_invasion	vascular invasion, lymphatic invasion, lymphovascular
flag_regression	regressive, regression, involutive
flag_nevus_component	melanocytic nevus, compound nevus, junctional nevus

4.4 Validation

Before fusion, the quality of embeddings is validated with two checks:

UMAP visualisation — embeddings projected to 2D. If BioClinicalBERT has captured class-relevant information, same-class patients should cluster together.

k-NN leave-one-out accuracy — each patient is classified by its nearest neighbour in embedding space. Accuracy above random baseline (33%) confirms the embeddings are discriminative.

5. Late Fusion

Late fusion combines the two modalities at the feature level — after each encoder has independently produced a fixed-length vector. This is contrasted with early fusion (combining raw inputs) or intermediate fusion (combining intermediate layer activations).

5.1 Why Late Fusion?

Strategy	When to use	Our case
Early fusion	Modalities tightly coupled, same format	Images + text are heterogeneous — not suitable
Intermediate	Shared transformer/attention across modalities	Models require joint architecture retraining
Late fusion	Independent strong encoders per modality	PanDerm + BioClinicalBERT are both strong individually

Late fusion also has a practical advantage: image and text pipelines can be developed and validated independently, and different patients can have one or both modalities available.

5.2 Fusion Mechanism

For each patient with both modalities, the image and text feature vectors are concatenated to form a joint representation:

Image features: (1, 1024) – PanDerm CLS token, mean-pooled across images
Text embeddings: (1, 768) – BioClinicalBERT mean-pooled tokens
Concatenated: (1, 1792) – joint multimodal representation

No learned fusion weights are needed. Concatenation is simple, interpretable, and avoids overfitting on a small dataset (138 matched patients). The logistic regression classifier then learns which dimensions of the 1792-dim vector are most predictive for each class.

5.3 Classification

A **Logistic Regression** classifier (scikit-learn) is trained on the fused features using the same 5-fold patient-level splits as the image pipeline. Only patients with both dermoscopy images and clinical reports are included (138 of 177 patients).

Classifier setting	Value
Algorithm	Logistic Regression (multinomial)
Solver	L-BFGS
Regularisation C	1.0
Max iterations	2000
Class weights	Balanced (compensates for class imbalance)
Random seed	42

A simple linear classifier is intentionally chosen over a neural network. With only 138 patients, a neural classifier would overfit. Logistic regression is also fully interpretable — the weights show which feature dimensions drive each class decision.

5.4 Results

Model	Balanced Acc	Macro AUC	Macro F1
PanDerm pre-trained (no fine-tune)	0.524 ± 0.066	0.723 ± 0.055	0.506 ± 0.065
PanDerm fine-tuned (image only)	0.546 ± 0.040	0.699 ± 0.041	0.523 ± 0.047
PanDerm + BioClinicalBERT (fusion)	0.603 ± 0.060	0.778 ± 0.057	0.576 ± 0.061

Per-class breakdown (fusion model):

Class	F1	AUC
Dysplastic Nevus	0.601 ± 0.131	0.785 ± 0.083
Melanoma Stage IA	0.722 ± 0.093	0.890 ± 0.069
Melanoma In Situ	0.404 ± 0.119	0.660 ± 0.111

Melanoma In Situ is the hardest class — it shares visual features with both Dysplastic Nevus (benign) and Melanoma Stage IA (invasive), and the class has the fewest patients. Fusion improves all three classes over image-only.

6. Evaluation & Outputs

All evaluation runs at patient level, not image level. For patients with multiple images, features are averaged before classification. This prevents inflated metrics from repeated images of the same lesion.

6.1 Metrics

Metric	Why chosen
Balanced Accuracy	Primary metric — handles class imbalance without assuming equal class sizes
Macro AUC	Area under ROC curve averaged across classes, threshold-independent
Macro F1	Harmonic mean of precision and recall, balanced across classes
Cohen's Kappa	Agreement beyond chance — important for clinical validation
Per-class F1 & AUC	Identify which specific classes the model struggles with

6.2 Plots Generated

Plot	Script	Description
confusion_matrix_aggregate.png	evaluate.py	Row-normalised % confusion matrix, summed across all folds
roc_curves_mean.png	evaluate.py	Mean ROC curve $\pm$ std band per class (one-vs-rest)
fold_accuracy_bars.png	evaluate.py	Balanced accuracy per fold vs. random baseline (33%)
umap_patient_features.png	evaluate.py	UMAP projection of prediction probabilities, coloured by class
attention_map_examples.png	evaluate.py	Original image vs. CLS attention overlay for 3 images per class

6.3 Attention Maps

Attention maps visualise which image regions the model attends to when producing the [CLS] token. They are extracted by hooking into the final transformer block's attention module.

Computation steps:

1. Register a forward hook on `model.blocks[-1].attn`.
2. Inside the hook, manually compute Q, K, V from the input using the attention layer's QKV projection.
3. Compute attention weights: $\text{softmax}(QK^T / \text{sqrt}(d)) \rightarrow \text{shape } (1, 16, 197, 197)$.
4. Extract row 0 of the CLS token: `attention[:, :, 0, 1:]` $\rightarrow (16, 196)$.
5. Average over 16 heads $\rightarrow (196,)$ $\rightarrow$ reshape to 14 $\times$ 14 spatial grid.
6. Resize to 224 $\times$ 224 and overlay as heatmap on the original image.
7. Lesion masking: suppress attention outside the skin lesion (pixels with luminance < 10 are treated as black background).

A hook is used instead of monkey-patching the forward method because PanDerm's attention forward signature includes `rel_pos_bias`, making it fragile to override directly.

7. Configuration & Running the Pipeline

All paths and hyperparameters are centralised in **config.py** at the repository root. This is the only file that needs to be edited when deploying on a new machine.

7.1 config.py — key variables

Variable	Description
DATA_ROOT	Directory of raw dermoscopy images (one sub-folder per class)
PANDERM_REPO	Path to cloned PanDerm repo (git clone https://github.com/SiyuanYan1/PanDerm.git)
CHECKPOINT_LARGE	Path to panderm_ll_data6_checkpoint-499.pth pretrained weights
PIPELINE_DIR	Working directory — all outputs (results/, features/, cross-fold-csv/) go here
CLINICAL_DIR	Directory where clinical_embeddings.npy will be saved
N_FOLDS	Number of cross-validation folds (default: 5)
BATCH_SIZE	Feature extraction batch size (default: 32)
LOGREG_C	Logistic regression regularisation (default: 1.0)

7.2 Running

Install dependencies:

```
pip install -r requirements.txt
```

Run the full pipeline:

```
python run_pipeline.py --stage all
```

Run without fine-tuning (pretrained features only):

```
python run_pipeline.py --stage all --no-finetune
```

Resume from where you left off:

```
python run_pipeline.py --stage all --skip-existing
```

Run individual stages:

```
python run_pipeline.py --stage prepare python run_pipeline.py --stage segment  
python run_pipeline.py --stage extract_features # GPU job python run_pipeline.py  
--stage evaluate # CPU job
```

Stage	GPU required	Approx. time
prepare	No	< 1 min
segment	No	~5 min (562 images)
make_csv	No	< 1 min
finetune	Yes (≥16 GB)	~30–60 min per fold

extract_features	Yes	~5 min per fold
evaluate	No	< 1 min
clinical_modality	No	~2–5 min
fusion	No	< 1 min